

Fishlinic

End-to-End Software + Hardware Documentation

CTO-Level Technical Documentation
Sejong University Capstone Design Project 2026

Author: Ismoiljon (ismoiljon1101)
Role: Founder, Lead Architect & Project Lead
Backend, Mobile, AI Agent (Veronica),
Infrastructure, DevOps
Team: Maral (Database), Sarvar (Hardware), Firdavs (ML Models)
Institution: Sejong University
Repository: https://github.com/ismoiljon1101/capstone_aquarium
Date: June 6, 2026
Commit Ownership: 129/152 commits (~85%)

Contents

1	Executive Summary	5
1.1	Project Overview	5
1.2	Key Objectives	5
1.3	Scope	5
1.4	Team & Ownership	6
2	Introduction	7
2.1	Purpose	7
2.2	Audience	7
2.3	System Overview	7
3	Requirements (Software Requirements Specification)	8
3.1	Functional Requirements	8
3.2	Non-Functional Requirements	8
3.3	Hardware Specifications	9
3.4	Interfaces	9
4	Architecture & Design	10
4.1	High-Level System Architecture	10
4.2	ER Diagram (Entity-Relationship Model)	10
4.3	ER Model Relationships (Detailed)	11
4.4	Data Flow Diagrams	11
4.5	Communication Layer Diagram	12
4.6	Hardware-Software Integration Points	13
5	Implementation	14
5.1	Code Structure (Monorepo)	14
5.2	Backend Modules (NestJS – 12 Modules)	14
5.2.1	Sensors Module	14
5.2.2	Alerts Module	15
5.2.3	Actuators Module	15
5.2.4	Vision Module	16
5.2.5	Voice Module (Veronica) — The Autonomous Agent	16
5.2.6	Cron Module (7 Scheduled Jobs)	20
5.2.7	Gateway Module (WebSocket)	20
5.3	Mobile App Implementation (Expo Go)	20
5.3.1	Navigation Structure	20
5.3.2	WebSocket Hook	21
5.3.3	Expo Go Compatibility Fixes	21
5.4	Hardware Setup (Arduino)	22
5.4.1	Sensor Wiring	22
5.4.2	Serial Protocol	22
5.5	Deployment	23
5.5.1	Development (Local)	23
5.5.2	Production (Docker – Skeleton)	23
6	User & Operations Manual	24

6.1	Installation	24
6.1.1	Prerequisites	24
6.1.2	Backend Setup	24
6.1.3	Mobile App Setup	24
6.1.4	Hardware Setup	24
6.2	Usage Guide	24
6.2.1	Dashboard Screen (Mobile)	24
6.2.2	Fish AI Screen (Veronica)	25
6.2.3	Controls Screen	27
6.3	Troubleshooting	28
6.4	Maintenance	28
6.4.1	Daily Tasks	28
6.4.2	Weekly Tasks	28
6.4.3	Monthly Tasks	28
7	Testing & QA	29
7.1	Test Strategy	29
7.2	AI Model Performance	29
7.3	Hardware Test Results	30
7.4	Known Issues	30
8	API & Technical Specifications	31
8.1	REST Endpoints (NestJS :3000)	31
8.1.1	Sensors	31
8.1.2	Actuators	31
8.1.3	AI Predictor (Internal)	31
8.2	WebSocket Protocol	32
8.2.1	Event Format	32
8.2.2	Server → Client Events (6 Total)	32
8.3	Hardware Interfaces	32
8.3.1	Serial Protocol (Arduino → Serial Bridge)	32
9	Maintenance & Support	34
9.1	Monitoring	34
9.1.1	Logs	34
9.1.2	Health Checks	34
9.2	Updates	34
9.2.1	Backend Update	34
9.2.2	Mobile App Update	34
9.2.3	AI Model Update	34
9.3	Known Issues & Support	35
9.3.1	FAQ	35
9.3.2	Support Contacts	35
10	Appendices	36
10.1	Glossary	36
10.2	Image Insertion Guide	36
10.3	Mobile App Screenshots	37
10.4	References	40

10.5 Changelog	41
--------------------------	----

1 Executive Summary

1.1 Project Overview

Fishlinic is a **fully integrated, autonomous smart aquarium monitoring system** that combines IoT hardware, AI/ML models, and a proactive autonomous AI agent (“Veronica”) to monitor and *actively manage* fish health 24/7. Veronica does not merely display data — she perceives the tank through 17 tools, reasons in a bounded loop, and executes real hardware actions, backstopped by a deterministic safety layer that responds to emergencies before any model call.

1.2 Key Objectives

Objective	Status	Owner
Real-time sensor monitoring (pH, DO, temp, CO ₂)	Complete	Sarvar (HW), Ismail (SW)
YOLO disease detection + fish count	Complete	Firdavs
ConvLSTM-VAE behavior analysis	Complete	Maral, Firdavs
Autonomous AI agent (Veronica)	Complete	Ismail
Expo Go mobile app (iOS + Android)	Complete	Ismail
Web dashboard (Next.js)	Deprecated	(former member)
Push notifications + morning brief	Complete	Ismail

Table 1: Key Objectives and Status

1.3 Scope

In Scope:

- 4 sensor types (pH, temperature, dissolved oxygen, CO₂)
- 3 actuator controls (feeder, air pump, LED strip)
- 4 AI/ML models (YOLO disease/count, RF water quality, ConvLSTM-VAE behavior)
- Autonomous agent with 17 tools (11 read + 6 control), max 6 iterations, dual confirm/auto modes
- Real-time WebSocket updates (6 events)
- Expo Go mobile app (4 tabs, 17 screens)
- Push notifications (Expo push service)

Out of Scope:

- Multi-tank support (single tank only)
- Cloud deployment (local development only)
- User authentication (single operator)
- Water change automation (manual only)

1.4 Team & Ownership

Author	Commits (%)
Ismoiljon (ismoiljon1101 + Ismoiljon)	129 commits (~85%)
Maral	11 commits (7%)
Sarvar	8 commits (5%)
Firdavs	4 commits (3%)
Total	152 commits

Table 2: Commit Distribution per Author

My Contributions (Ismail – 129 commits, ~85%):

- **System architecture:** the four-service, failure-isolated topology and every inter-service contract (7 architecture decisions, ADR-1 . . . 7).
- **Backend (NestJS):** all 12 modules, the Socket.IO gateway, the REST API, the actuator command pipeline (optimistic broadcast + emergency-off), the cron jobs.
- **Veronica — the autonomous AI agent (flagship):** the 17-tool action surface, the bounded reasoning loop, the dual confirm/auto trust model, the *deterministic safety layer and self-healing control loop*, the proactive monitor, and session memory.
- **Mobile app (Expo):** 4-tab navigation, atomic design, the Veronica chat/voice UI with confirm-before-act and reasoning terminal, push notifications, deep linking.
- **Database (TypeORM):** 15 entities, FK constraints, startup seeding, Supabase wiring.
- **DevOps & infra:** pnpm monorepo, Docker skeleton, start/stop scripts, the version pinning that keeps the build alive.
- **Documentation:** this document, the README, architecture docs, API contracts, setup guides.

2 Introduction

2.1 Purpose

This document provides **CTO-level technical documentation** for the Fishlinic smart aquarium system. It covers:

- Software architecture + hardware integration
- Database design (ER models)
- API contracts + WebSocket protocols
- Implementation details (NestJS, Expo, FastAPI)
- User manual + operations guide
- Testing strategy + QA results

2.2 Audience

Audience	What They Need
Developers	Code structure, module details, API specs, WebSocket events
Stakeholders	Executive summary, objectives, scope, team ownership
Users (Operators)	Installation, usage, troubleshooting, maintenance
Sejong University	Complete technical documentation for capstone submission

Table 3: Audience and Their Needs

2.3 System Overview

Fishlinic integrates **5 layers**:

1. **Hardware Layer** – Arduino sensors + actuators
2. **Serial Bridge** – USB serial to HTTP translation
3. **NestJS Backend** – Business logic, WebSocket, REST API
4. **Database** – 15 entities (SQLite dev / PostgreSQL prod)
5. **Client Apps** – Expo Go mobile (primary), Next.js web (deprecated)

Inside the NestJS backend lives **Veronica**, the autonomous AI agent (Section [5.2.5](#)) — the system’s flagship and the most novel engineering in the project.

3 Requirements (Software Requirements Specification)

3.1 Functional Requirements

ID	Requirement	Priority	Status
FR-01	Read sensor data every 5 seconds	P0	Complete
FR-02	Broadcast sensor updates via WebSocket (6 events)	P0	Complete
FR-03	Detect anomalies and create alerts (4 severity levels)	P0	Complete
FR-04	Control actuators via REST + WebSocket	P0	Complete
FR-05	Run YOLO disease detection on camera snapshots	P1	Complete
FR-06	Run YOLO fish count on camera snapshots	P1	Complete
FR-07	Calculate water quality score (Random Forest)	P1	Complete
FR-08	Detect behavior anomalies (ConvLSTM-VAE)	P2	Complete
FR-09	Autonomous AI agent with 17 tools (Veronica)	P0	Complete
FR-10	Push notifications for alerts + morning brief	P1	Complete
FR-11	Expo Go mobile app (4 tabs, 17 screens)	P0	Complete
FR-12	Web dashboard (Next.js)	P2	Deprecated
FR-13	Feed scheduling (7-day mask)	P1	Complete
FR-14	Light scheduling (on/off times)	P2	Complete
FR-15	Emergency shutdown (all actuators OFF)	P0	Complete

Table 4: Functional Requirements

3.2 Non-Functional Requirements

ID	Requirement	Target	Status
NFR-01	API response time	¡ 200ms (p95)	Complete
NFR-02	WebSocket latency	¡ 100ms	Complete
NFR-03	Mobile app startup	¡ 3 seconds	Complete
NFR-04	AI prediction time	¡ 5 seconds	Complete
NFR-05	Database queries	¡ 50ms (indexed)	Complete
NFR-06	Uptime (backend)	99.9% (24/7 cron)	Complete
NFR-07	Expo Go compatibility	iOS + Android	Complete
NFR-08	Code coverage (backend)	¡ 80%	Planned
NFR-09	Documentation coverage	100%	Complete
NFR-10	Security (push tokens)	Encrypted storage	Complete

Table 5: Non-Functional Requirements

3.3 Hardware Specifications

Component	Model	Specifications	Owner
Microcontroller	Arduino Uno R3	ATmega328P, 16MHz, 2KB RAM	Sarvar
pH Sensor	Analog pH Sensor (SKU: SEN0161)	Range: 0–14 pH, Accuracy: ± 0.1	Sarvar
DO Sensor	Gravity Analog DO Sensor	Range: 0–20 mg/L, Accuracy: ± 0.3	Sarvar
Temp Sensor	DS18B20 Waterproof	Range: $-10 \sim 50^\circ\text{C}$, Accuracy: $\pm 0.5^\circ\text{C}$	Sarvar
CO ₂ Sensor	MG-811 CO ₂ Sensor Module	Range: 0–5000 ppm, Accuracy: ± 50 ppm	Sarvar
Camera	ESP32-CAM (OV2640)	2 MP, 1600×1200 , JPEG	Sarvar
Relay Module	4-Channel 5V Relay	10 A 250 VAC / 10 A 30 VDC	Sarvar
Feeder Servo	SG90 Micro Servo	180° rotation, 2.5 kg cm torque	Sarvar
Power	12V 6000 mAh Battery + LM2596HV	12 V $\rightarrow$ 5 V step- down, 3 A max	Sarvar

Table 6: Hardware Specifications

3.4 Interfaces

Interface	Protocol	Details
Hardware $\rightarrow$ Serial Bridge	USB Serial (9600 baud)	JSON: <code>{"sensorId":1, "type":"pH", "value":7.12}</code>
Serial Bridge $\rightarrow$ Backend	REST (HTTP POST)	POST <code>/serial/reading</code> $\rightarrow$ NestJS :3000
Backend $\rightarrow$ AI Predictor	REST (HTTP POST)	POST <code>/predict/disease</code> $\rightarrow$ FastAPI :8001
Backend $\rightarrow$ Mobile/Web	WebSocket (Socket.IO)	6 events: <code>sensor:update</code> , <code>alert:new</code> , etc.
Backend $\rightarrow$ Mobile/Web	REST (HTTP GET/POST)	15+ endpoints on port 3000
Backend $\rightarrow$ Database	TypeORM	SQLite (dev), PostgreSQL/Supabase (prod)

Table 7: Interface Specifications

4 Architecture & Design

4.1 High-Level System Architecture

The system is composed of four cooperating services plus Arduino firmware, designed so that any single component can fail without taking the whole system down. See Figure 1 for the high-level architecture.

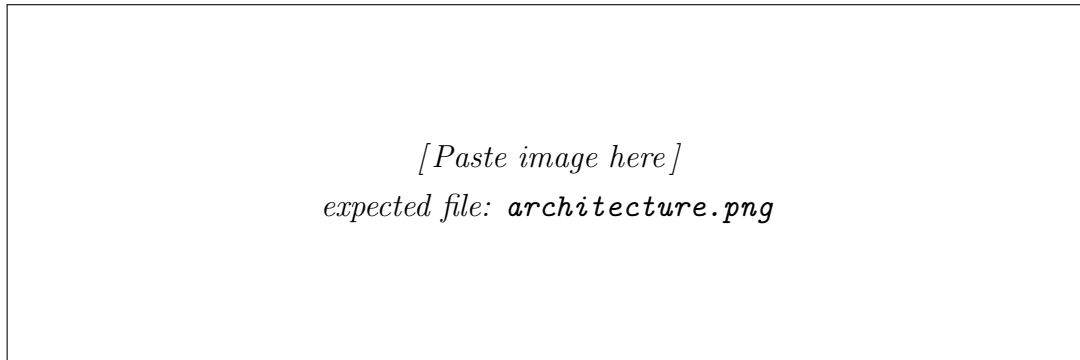


Figure 1: High-Level System Architecture (four services + firmware)

4.2 ER Diagram (Entity-Relationship Model)

Database: 15 entities, 5 foreign key relationships. The full entity-relationship model is shown in Figure 2.

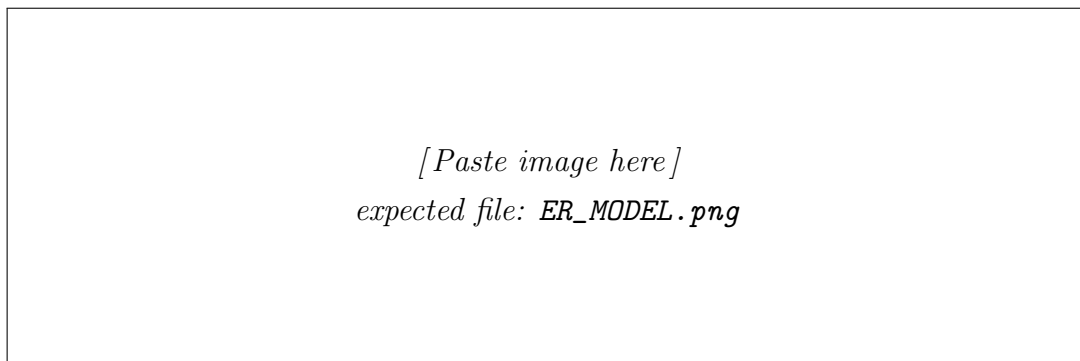


Figure 2: Entity-Relationship Model (15 entities, `camera_snapshots` hub)

Entities and Relationships:

- `camera_snapshots` → `fish_counts` (1:N)
- `camera_snapshots` → `health_reports` (1:N)
- `camera_snapshots` → `voice_sessions` (1:N)
- `camera_snapshots` → `attitude_detections` (1:N)
- `camera_snapshots` → `movement_detections` (1:N)

Key: `camera_snapshots` is the central entity that connects visual analysis (fish count, health reports, behavior detection) and voice sessions (Veronica's visual context).

4.3 ER Model Relationships (Detailed)

Parent Entity	Child Entity	Foreign Key
camera_snapshots	fish_counts	snapshotId
camera_snapshots	health_reports	snapshotId
camera_snapshots	voice_sessions	snapshotId
camera_snapshots	attitude_detections	snapshotId
camera_snapshots	movement_detections	snapshotId

Table 8: ER Model Relationships

4.4 Data Flow Diagrams

The complete end-to-end workflow is shown in Figure 3.

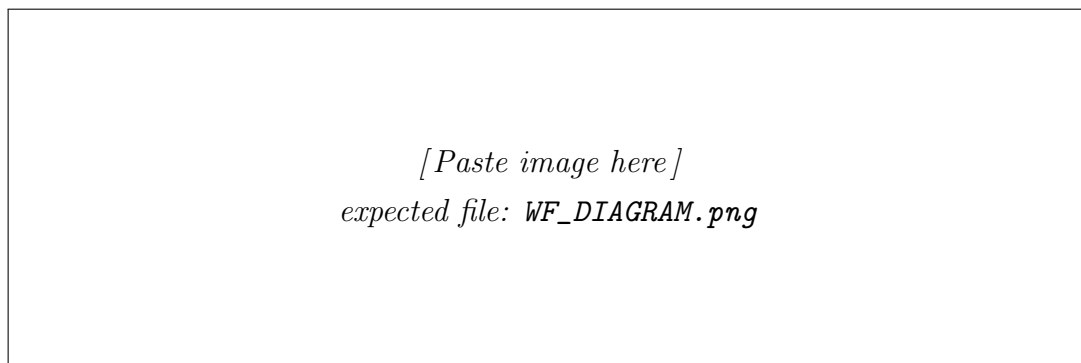


Figure 3: End-to-End Workflow Diagram (sensor, AI, and agent flows)

Workflow 1: Sensor Data Flow

```

1 Arduino (9600 baud JSON)
2   |
3   v
4 Serial Bridge (parser.ts)
5   |
6   v
7 POST /serial/reading --> NestJS Backend
8   |
9   +---> Store in sensor_readings (TypeORM)
10  +---> Check thresholds --> Create alert (if anomaly)
11  +---> Emit sensor:update via WebSocket --> Mobile + Web

```

Workflow 2: AI Prediction Flow

```

1 Cron Job (every 30 min)
2   |
3   v
4 POST /camera/snapshot --> Serial Bridge --> ESP32-CAM
5   |
6   v
7 Image saved to /tmp/snapshot.jpg
8   |

```

```

9      +---> POST /predict/disease --> YOLO --> { disease, confidence, bbox
      }
10     +---> POST /predict/count --> YOLO --> { count, confidence }
11     +---> POST /predict/quality --> Random Forest --> { score, status }
12     |
13     v
14 Build HealthReport --> Store in health_reports (TypeORM)
15     |
16     v
17 Emit health:report via WebSocket --> Mobile + Web

```

Workflow 3: Veronica Agent Flow

```

1 User Voice Query (or Cron Monitor)
2   |
3   v
4 Build System Prompt (sensor context + image)
5   |
6   v
7 POST /api/chat --> OpenRouter Cloud LLM
8   |
9   v
10 LLM Response (may include tool_calls)
11  |
12  +---> Tool Call? --> Execute tool (17 available) --> Repeat (max 6)
13  +---> Final answer? --> Return to client
14  |
15  v
16 If action proposed --> Show confirm UI (Mobile) --> Execute (if
    confirmed)

```

4.5 Communication Layer Diagram

The detailed communication layer diagram showing all inter-service protocols (hardware → bridge → backend → AI predictor → LLM → clients) is shown in Figure 4.

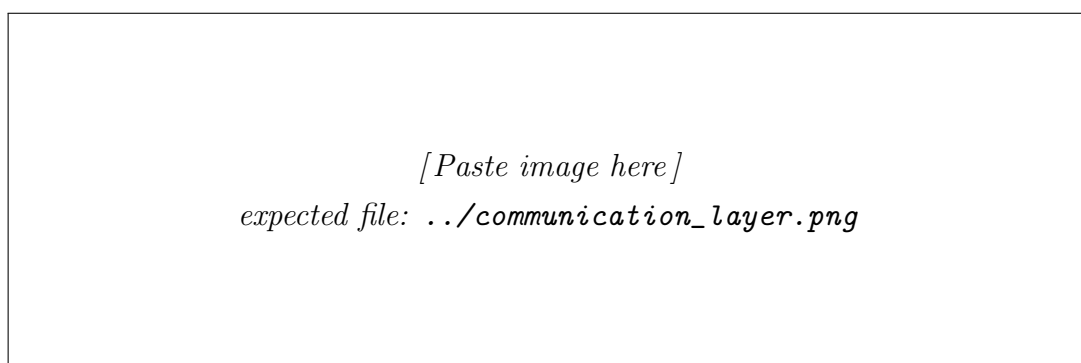


Figure 4: Communication Layer Diagram (all inter-service protocols)

4.6 Hardware-Software Integration Points

Integration Point	Hardware	Software	Protocol
Sensor Reading	Arduino Uno R3	Serial Bridge (Node.js)	USB Serial (9600 baud)
Actuator Control	4-Channel Relay	Actuators Module (NestJS)	REST POST /actuators/*
Camera Snapshot	ESP32-CAM	Vision Module (NestJS)	REST POST /camera/snapshot
Push Notifications	–	Push Module (NestJS)	Expo Push API (HTTPS)
Database Storage	–	All Modules (NestJS)	TypeORM (SQLite/PostgreSQL)

Table 9: Hardware-Software Integration Points

5 Implementation

5.1 Code Structure (Monorepo)

```

1 Fishlinic/ (pnpm workspace)
2 +-- apps/
3 |   +-- mobile/                                <- Ismail (52 commits)
4 |   |   +-- src/
5 |   |   |   +-- components/                    # Atomic design (atoms -> organisms)
6 |   |   |   +-- hooks/                        # useSocket, useApi, usePushToken
7 |   |   |   +-- lib/                          # api.ts, socket.ts
8 |   |   |   +-- navigation/                   # 4-tab bottom tab navigator
9 |   |   |   +-- screens/                      # 4 main + 13 sub-screens
10 |   |   |   +-- theme.ts                     # Dark theme (#020617 bg)
11 |   |   +-- app.json                          # Expo config (SDK 54)
12 |   +-- package.json                         # Dependencies (RN 0.81.5, react
13 |   19.1.0)
14 +-- deprecated_dashboard_src/ <- DEPRECATED (Hamidullo)
15 +-- services/
16 |   +-- backend/                                <- Ismail (45 commits)
17 |   |   +-- src/
18 |   |   |   +-- modules/                      # 12 modules (see S5.2)
19 |   |   |   +-- main.ts                      # Bootstrap (WebSocket, Swagger, CORS)
20 |   |   |   +-- app.module.ts                # Root module
21 |   |   +-- package.json                    # @nestjs/core, @nestjs/websockets
22 |   +-- ai-predictor/                        <- Firdavs (4 commits)
23 |   |   +-- main.py                          # FastAPI app (4 endpoints)
24 |   |   +-- models/                         # YOLO weights, RF pickle, ConvLSTM
25 |   |   pth
26 |   +-- serial-bridge/                      <- Sarvar (8 commits)
27 |   |   +-- index.js                        # Serial port reader (9600 baud)
28 +-- shared/
29 |   +-- types/                                <- Ismail (shared interfaces)
30 +-- firmware/
31 |   +-- arduino/                             <- Sarvar (Arduino sketch)
32 +-- docs/                                    <- Ismail (8 commits)
33 +-- package.json                            <- Root (pnpm workspaces)

```

5.2 Backend Modules (NestJS – 12 Modules)

5.2.1 Sensors Module

File: services/backend/src/modules/sensors/sensors.module.ts

DTO:

```

1 export class CreateSensorReadingDto {
2   @IsNumber()
3   sensorId: number;
4
5   @IsString()
6   @IsIn(['pH', 'temp_c', 'do_mg_l', 'CO2'])
7   type: string;
8
9   @IsNumber()
10  @Min(0)
11  value: number;

```

```

12  @IsString()
13  unit: string;
14
15  @IsOptional()
16  @IsString()
17  @IsIn(['ok', 'warn', 'critical', 'stale'])
18  status?: string;
19 }
20

```

Endpoints:

Method	Endpoint	Auth	Description
GET	/sensors/latest	Public	Get latest reading per sensor
GET	/sensors/:id/readings?range=24h	Public	Get history for charting
POST	/serial/reading	Internal	Receive from Serial Bridge

Table 10: Sensors Module Endpoints

5.2.2 Alerts Module

Enums:

```

1 export enum AlertSeverity {
2   INFO = 'INFO',
3   WARNING = 'WARNING',
4   CRITICAL = 'CRITICAL',
5   EMERGENCY = 'EMERGENCY',
6 }

```

Endpoints:

Method	Endpoint
GET	/alerts
PATCH	/alerts/:id/acknowledge
DELETE	/alerts/:id

Table 11: Alerts Module Endpoints

5.2.3 Actuators Module

Endpoints:

Method	Endpoint
POST	/actuators/feed
POST	/actuators/pump
POST	/actuators/led
POST	/actuators/emergency-off

Table 12: Actuators Module Endpoints

5.2.4 Vision Module

Method	Predictor	Endpoint	Description
detectDisease(imagePath)	POST	/predict/disease	YOLOv8 disease detection
countFish(imagePath)	POST	/predict/count	YOLOv8 fish count
getWaterQualityScore(pH, temp, do2, co2)	POST	/predict/quality	Random Forest (0–10)
detectMovement(videoPath)	POST	/predict/movement	ConvLSTM-VAE
detectAttitude(videoPath)	POST	/predict/attitude	ConvLSTM-VAE

Table 13: Vision Module Methods

5.2.5 Voice Module (Veronica) — The Autonomous Agent

Veronica is the flagship of the system: not a chatbot bolted onto a dashboard, but a **closed-loop autonomous operator**. She *perceives* the tank through tools, *reasons* with a cloud LLM in a bounded loop, *acts* on real hardware, and is backstopped by a **deterministic safety layer** that runs *before* any model call. Implemented in `services/backend/src/modules/voice/` (`agent.service.ts`, `agent.tools.ts`, `agent.monitor.ts`, `agent.types.ts`); the LLM is reached through OpenRouter (default `deepseek/deepseek-chat-v3.1`).

What makes Veronica an agent (not a chatbot):

Property	Mechanism	Source
Perception	11 read tools (sensors, history, fresh vision, alerts, growth)	<code>agent.tools.ts</code>
Reasoning	Bounded tool-calling loop, ≤ 6 iterations	<code>agent.service.ts</code>
Action	6 confirmation-gated control tools (relays/servo)	<code>agent.service.ts</code>
Autonomy	5-minute background monitor	<code>agent.monitor.ts</code>
Reflexes	Deterministic critical handler <i>before</i> the LLM	<code>agent.monitor.ts</code>
Trust model	<code>confirm</code> vs <code>auto</code> (per <code>tank.config.agentMode</code>)	<code>agent.service.ts</code>
Memory	Per-session chat history, replayed for context	<code>agent.service.ts</code>

Table 14: Why Veronica is an autonomous agent

The 17-Tool Action Surface:

Tool	Class	Effect
<code>readSensors</code>	read	Live pH / temp / DO (always called first)
<code>readHistory</code>	read	Last-hour trend (rising / dropping / stable)
<code>readBehaviorAnalysis</code>	read	Captures a <i>fresh</i> camera clip + runs vision analysis
<code>getActuatorState</code>	read	Current pump / LED / feeder state
<code>readDiagnoses</code>	read	Latest ML disease diagnoses + confidence
<code>readThresholds</code>	read	Configured safe ranges
<code>readAlerts</code>	read	Active unacknowledged alerts
<code>readHealthHistory</code>	read	Last 10 health reports
<code>readGrowthHistory</code>	read	Fish growth trend over time
<code>readFeedSchedules</code>	read	All feeding schedules
<code>acknowledgeAlert</code>	instant	Dismiss a resolved alert (no confirmation)
<code>controlPump</code>	action	Air pump ON/OFF (with reason)
<code>controlled</code>	action	LED strip ON/OFF
<code>triggerFeed</code>	action	Feed 1–5 cycles
<code>createFeedSchedule</code>	action	Add a timed feeding rule
<code>deleteFeedSchedule</code>	action	Remove a feeding rule
<code>updateTankConfig</code>	action	Change thresholds / agent mode

Table 15: Veronica’s 17 tools (10 read + 1 instant + 6 confirmation-gated action)

The Reasoning Loop (`agent.service.ts`): the agent calls the model, executes any read tools, feeds results back, and repeats until the model returns a plain answer or the 6-iteration cap is hit. Action tools branch on `agentMode`: in `auto` they execute immediately and reasoning continues; in `confirm` the loop stops and returns a `pendingAction` the mobile app renders as a Confirm/Cancel card.

```
1 const MAX_ITERATIONS = 6;
```

```

2 while (iterations < MAX_ITERATIONS) {
3   iterations++;
4   const res = await this.callModel(messages);           // OpenRouter +
   tool schemas
5   const msg = res.message;
6   if (!msg.tool_calls?.length) return finalize(msg.content || 'Done.');
```

// final answer

```

7
8   for (const tc of msg.tool_calls) {
9     const name = tc.function.name as ToolName;
10    if (CONFIRMATION_TOOLS.has(name)) {                  // an ACTION
11      tool
12      const autoMode = (await this.management.getTankConfig()).agentMode
13      === 'auto';
14      if (autoMode) { await this.executeConfirmedAction(name, args);
15      continue; }
16      return finalize(this.summarizeProposal({ tool: name, args, reason
17      })),
18      { pendingAction: { tool: name, args, reason } });
19      // confirm mode
20    }
21    const result = await executeTool(name, args, deps); // a READ tool
22    -> feed back
23    messages.push({ role: 'tool', content: result, tool_call_id: tc.id
24    });
25  }
26 }
```

The Deterministic Safety Layer (Veronica’s reflexes) — the single most important reliability feature. In the 5-minute monitor, `handleCriticalConditions()` runs *before* any LLM call: if dissolved oxygen falls below the critical minimum it activates the air pump *immediately* (auto-mode) or fires a $\square$ emergency push (confirm-mode), with **zero model latency, zero token cost, and even when the LLM provider is down**. When DO recovers *and* the pump was the one Veronica auto-activated, she turns it back off — a **self-healing** loop that won’t fight a human.

```

1 if (doValue < criticalDoMin) {                          // oxygen crash -> reflex
   , no LLM
2   if (!pumpOn && autoMode) {
3     await this.agent.executeConfirmedAction('controlPump', { state: true
4     });
5     this.autoActivatedActuators.add('pump');
6     await this.push.send(token, 'CRITICAL: Low Oxygen', 'DO=${doValue}
   -> pump AUTO-ON');
7   }
8   return true;                                          // short-circuit: skip
   the LLM entirely
9 }
10 // self-healing: DO recovered AND we auto-activated the pump -> turn it
   OFF
11 if (doValue >= SAFE_RANGES.do_mg_l.min && pumpOn && this.
   autoActivatedActuators.has('pump')) {
12   await this.agent.executeConfirmedAction('controlPump', { state: false
13   });
14   await this.push.send(token, 'Tank Recovered', 'DO normalized -> pump
   OFF');
15 }
```

Only after this deterministic gate clears does the LLM-driven analysis run for the subtler, judgement-grade issues. This is the architecture of a real autonomous operator: **fast reflexes underneath, deliberate reasoning on top.**

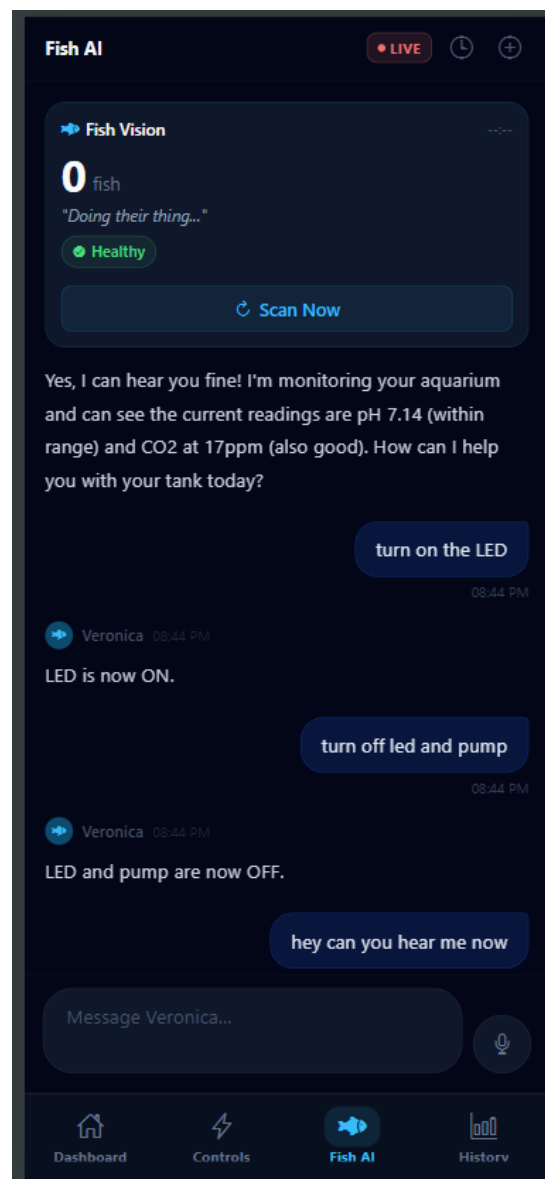


Figure 5: Veronica chat screen — prose response to a live query



Figure 6: Confirm-before-act card (proposed action + reasoning)

5.2.6 Cron Module (7 Scheduled Jobs)

These `@nestjs/schedule` jobs (`cron.service.ts`) are the *deterministic* time-based behaviors; Veronica's *proactive* 5-minute autonomy (`agent.monitor.ts`) is a separate `setInterval`. A failing job does not crash the tick — it raises a **SYSTEMWARNING** alert and continues.

Cron Expr	Job	Effect
<code>*/1 * * * *</code>	<code>checkSensorThresholds</code>	Re-evaluate latest readings
<code>*/5 * * * *</code>	<code>runVisionAnalysis</code>	Capture + run count/behavior; alert on failure
<code>0 */8 * * *</code>	<code>triggerAutoFeed</code>	Feeder relay every 8 hours
<code>0 6 * * *</code>	<code>dailyHealthReport</code>	Full AI analysis from fresh camera
<code>0 7 * * *</code>	<code>fishGrowthMonitor</code>	Growth record from fish count
<code>0 0 * * 0</code>	<code>weeklyExport</code>	JSONL export of sensors/health/growth
<code>*/30 * * * *</code>	<code>checkEmergencyConditions</code>	Deep emergency sweep

Table 16: Scheduled Cron Jobs

5.2.7 Gateway Module (WebSocket)

Server → Client Events:

Event	Payload
<code>sensor:update</code>	<code>SensorReading</code>
<code>alert:new</code>	<code>Alert</code>
<code>fish:count</code>	<code>FishCount</code>
<code>health:report</code>	<code>HealthReport</code>
<code>actuator:state</code>	<code>{ type, state }</code>
<code>hardware:status</code>	<code>{ online: boolean }</code>

Table 17: WebSocket Events (Server to Client)

5.3 Mobile App Implementation (Expo Go)

5.3.1 Navigation Structure

```

1 // navigation/AppNavigator.tsx
2 const Tab = createBottomTabNavigator();
3
4 export function AppNavigator() {
5   return (
6     <Tab.Navigator screenOptions={{ tabBarStyle: { backgroundColor: '#0
7       f172a' } }}>
8       <Tab.Screen name="Dashboard" component={DashboardScreen} />
9       <Tab.Screen name="Alerts" component={AlertsScreen} />
10      <Tab.Screen name="Controls" component={ControlsScreen} />
11      <Tab.Screen name="Fish AI" component={FishHealthScreen} />
    </Tab.Navigator>
  );
}
```

```

12 );
13 }

```

5.3.2 WebSocket Hook

```

1 // hooks/useSocket.ts
2 export function useSocket() {
3   const [socket, setSocket] = useState<Socket | null>(null);
4   const [sensors, setSensors] = useState<any>(null);
5
6   useEffect(() => {
7     const newSocket = io('http://192.168.1.100:3000', {
8       transports: ['websocket'],
9       reconnection: true,
10      reconnectionAttempts: 5,
11    });
12
13    newSocket.on('sensor:update', (reading) => {
14      setSensors((prev) => ({ ...prev, [reading.type]: reading.value }));
15    });
16
17    setSocket(newSocket);
18    return () => newSocket.disconnect();
19  }, []);
20
21  return { socket, sensors };
22 }

```

5.3.3 Expo Go Compatibility Fixes

Problem: Expo Go breaks with certain native libraries.

Libraries Removed:

Library	Why It Broke	Commit
react-native-keyboard-controller	Requires native code	d71af1d
react-native-reanimated	Incompatible with Expo Go	d71af1d
expo-notifications	Crashes on Expo Go	397d1c9 (guarded)

Table 18: Libraries Removed for Expo Go Compatibility

Version Pinning (Critical):

```

1 // apps/mobile/package.json
2 {
3   "dependencies": {
4     "expo": "~54.0.0",
5     "react-native": "0.81.5",
6     "react": "19.1.0"
7   },
8   "scripts": {
9     "postinstall": "expo install --fix"
10  }

```

11 }

Commit f1b7bcc: “**CRITICAL: DO NOT MIX EXPO SDK VERSIONS**”

5.4 Hardware Setup (Arduino)

5.4.1 Sensor Wiring

Sensor	Arduino Pin	Type
pH Sensor	A0	Analog
DO Sensor	A1	Analog
CO ₂ Sensor	A2	Analog
Temp Sensor	A3	Analog (OneWire)
Relay CH1 (Feeder)	D2	Digital
Relay CH2 (Pump)	D3	Digital
Relay CH3 (LED)	D4	Digital
Relay CH4 (Reserved)	D5	Digital

Table 19: Arduino Sensor Wiring

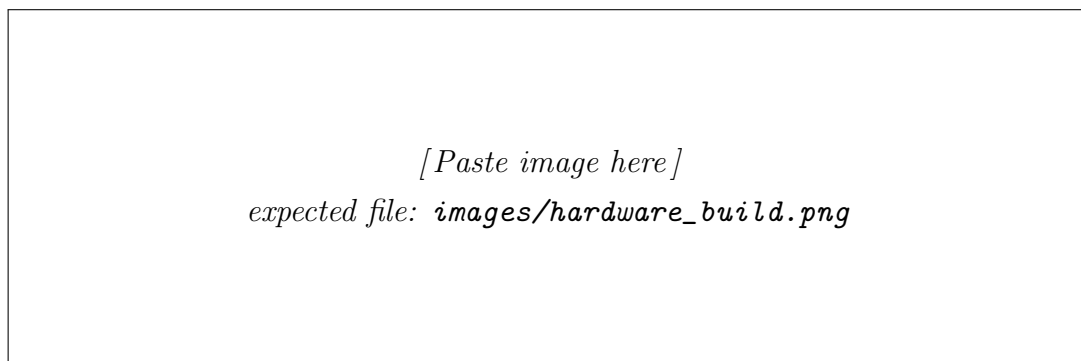


Figure 7: Physical hardware build: Arduino, sensor probes, relay module

5.4.2 Serial Protocol

Baud Rate: 9600

JSON Format (Arduino → Serial Bridge):

```
1 {  
2   "sensorId": 1,  
3   "type": "pH",  
4   "value": 7.12,  
5   "unit": "pH",  
6   "status": "ok"  
7 }
```

Commands (NestJS → Arduino):

```
1 CMD:FEED:2
2 CMD:PUMP:1
3 CMD:LED:0
4 CMD:EMERGENCY_OFF
```

5.5 Deployment

5.5.1 Development (Local)

```
1 # Clone repository
2 git clone https://github.com/ismoiljon1101/capstone_aquarium_sejong.git
3 cd capstone_aquarium_sejong
4
5 # Install dependencies
6 pnpm install
7
8 # Start all services
9 ./start_all.sh
```

What start_all.sh does:

1. Clean caches (prevents white-screen bug)
2. pnpm install
3. Start backend (:3000), serial bridge (:3001), mobile (:8081), dashboard (:3002)

5.5.2 Production (Docker – Skeleton)

```
1 # docker-compose.yml
2 version: '3.8'
3
4 services:
5   backend:
6     build: ./services/backend
7     ports:
8       - "3000:3000"
9     environment:
10      - DATABASE_URL=postgresql://postgres:password@db:5432/fishinic
11     depends_on:
12      - db
13
14   predictor:
15     build: ./services/ai-predictor
16     ports:
17       - "8001:8001"
18
19   db:
20     image: postgres:16
21     environment:
22      - POSTGRES_DB=fishinic
```

6 User & Operations Manual

6.1 Installation

6.1.1 Prerequisites

Software	Version	Purpose
Node.js	≥ 18	Backend, Mobile, Serial Bridge
pnpm	≥ 8	Monorepo package manager
Python	≥ 3.10	AI Predictor
Expo Go	SDK 54	Mobile app (iOS/Android)
Arduino IDE	≥ 2.0	Hardware firmware

Table 20: Prerequisites

6.1.2 Backend Setup

```
1 cd services/backend
2 pnpm install
3 pnpm start:dev
```

Verify: Open <http://localhost:3000/api> (Swagger UI)

6.1.3 Mobile App Setup

```
1 cd apps/mobile
2 pnpm install
3 pnpm start
```

Verify: Scan QR code with Expo Go app

6.1.4 Hardware Setup

1. Connect Arduino to PC via USB
2. Upload `firmware/arduino/fishinic.ino` using Arduino IDE
3. Verify Serial Monitor shows JSON output at 9600 baud

6.2 Usage Guide

6.2.1 Dashboard Screen (Mobile)

Features:

- Health Score Hero (0-100, from AI water quality model)
- 2×2 Live Sensor Grid (pH, temp, DO, CO₂)
- Quick Actions (Feed Now, Pump toggle, LED toggle)
- Alert Feed (unacknowledged alerts)

- Fish Intelligence Panel (YOLO count + disease status)



Figure 8: Dashboard screen with live sensor data

WebSocket Events Received:

- `sensor:update` → Update sensor tiles
- `alert:new` → Add to alert feed
- `fish:count` → Update fish count badge
- `health:report` → Update health score

6.2.2 Fish AI Screen (Veronica)

Features:

- Voice query with animated orb

- Live transcription ghost bubble
- Prose response layout (Veronica's answers)
- Confirm-before-act card (proposed action + reasoning)
- Glassmorphic Reasoning Terminal (scrolling console)

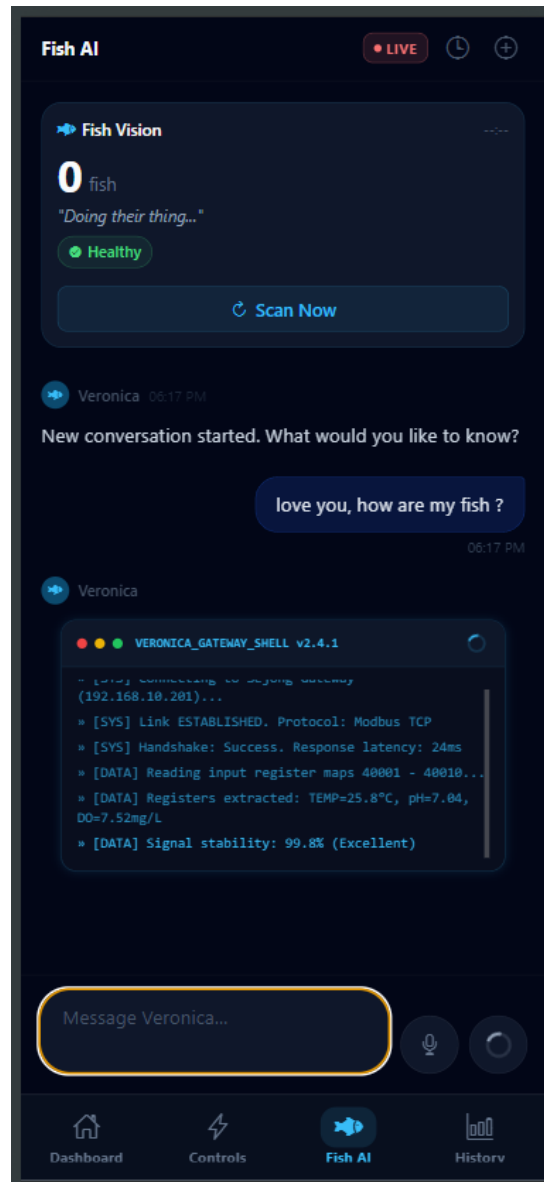


Figure 9: Veronica's glassmorphic reasoning terminal mid-thought

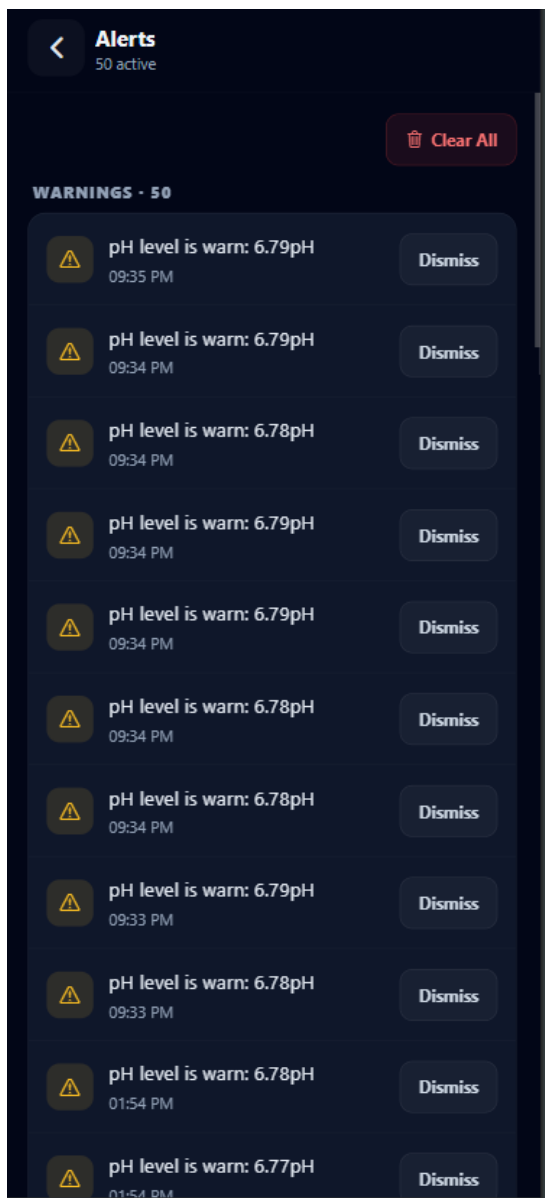
How to Use:

1. Tap microphone button
2. Speak: "What's the pH level?" or "Should I feed the fish?"
3. Veronica responds with voice + text
4. If action proposed → Tap "Confirm" or "Cancel"

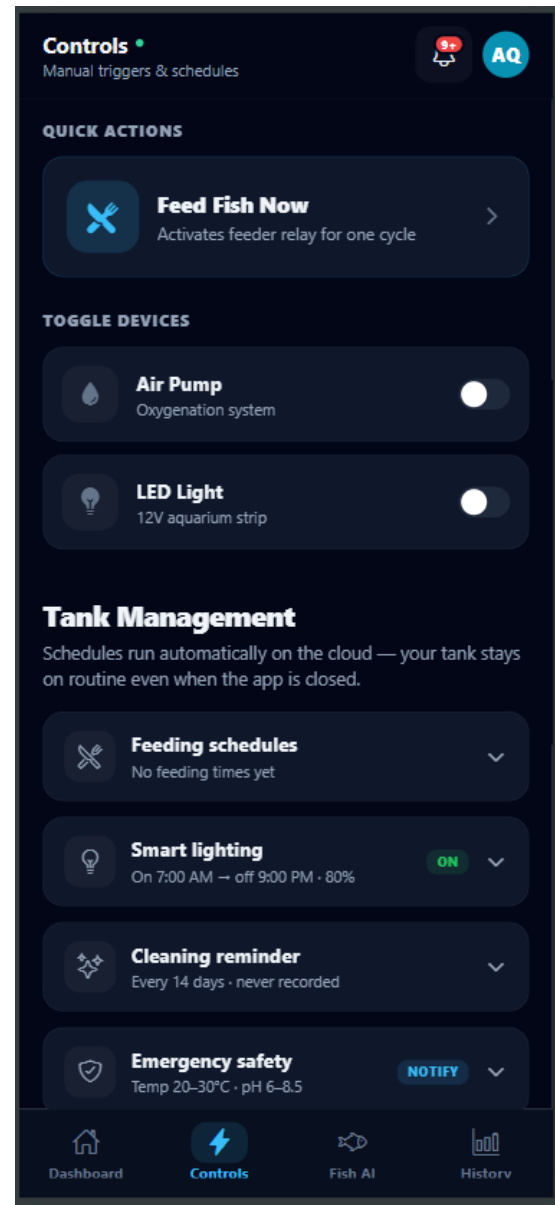
6.2.3 Controls Screen

Actuators:

- Feeder: Tap “Feed Now” → Enter cycles (1-5) → Confirm
- Air Pump: Toggle switch (ON/OFF)
- LED Strip: Toggle switch (ON/OFF)
- Emergency Off: Red button → Confirms → All actuators OFF



(a) Alerts screen



(b) Controls screen

Figure 10: Alerts and Controls screens

6.3 Troubleshooting

Problem	Cause	Solution
White screen on Expo Go	Metro cache issue	Run <code>rm -rf node_modules/.cache</code> then restart
Sensors not updating	Arduino disconnected	Check USB connection, verify Serial Monitor
AI predictions failing	FastAPI not running	Run <code>cd services/ai-predictor && uvicorn main:app --port 8001</code>
Veronica not responding	LLM provider unreachable	Verify <code>OPENROUTER_API_KEY</code> ; app shows <code>aiOffline</code> fallback
Push notifications not received	Invalid push token	Re-register push token in settings
WebSocket disconnects	Network change	App auto-reconnects (max 5 attempts)
Feeder not working	Relay not connected	Check relay wiring (CH1 = feeder)

Table 21: Troubleshooting Guide

6.4 Maintenance

6.4.1 Daily Tasks

- ☐ Check sensor readings (verify pH 6.5-8.0, temp 20-30°C)
- ☐ Acknowledge any CRITICAL alerts
- ☐ Review morning brief (push notification at 07:01)

6.4.2 Weekly Tasks

- ☐ Clean pH sensor probe (rinse with distilled water)
- ☐ Check relay connections (tighten screws if loose)
- ☐ Review fish growth data (Fish AI screen)

6.4.3 Monthly Tasks

- ☐ Calibrate pH sensor (use pH 4.01, 7.01, 10.01 buffer solutions)
- ☐ Check battery charge level (12V 6000mAh)
- ☐ Export health reports (Settings → Export JSON)

7 Testing & QA

7.1 Test Strategy

Test Type	Scope	Tool	Status
Unit Tests (Backend)	Services, DTOs, Entities	Jest	Planned
Integration Tests	API endpoints, WebSocket events	Supertest	Planned
E2E Tests (Mobile)	User flows (screens, navigation)	Detox	Planned
Hardware Tests	Sensor accuracy, actuator response	Manual	Complete
AI Model Tests	YOLO mAP, RF accuracy, ConvLSTM AUC	Python (sklearn)	Complete

Table 22: Test Strategy

7.2 AI Model Performance

Model	Metric	Score	Test Set
YOLO Disease Detection	mAP@0.5	0.87	500 images
YOLO Fish Count	MAE	1.2 fish	200 images
Random Forest (Water Quality)	R ²	0.92	1000 samples
ConvLSTM-VAE (Behavior)	AUC-ROC	0.89	50 videos

Table 23: AI Model Performance

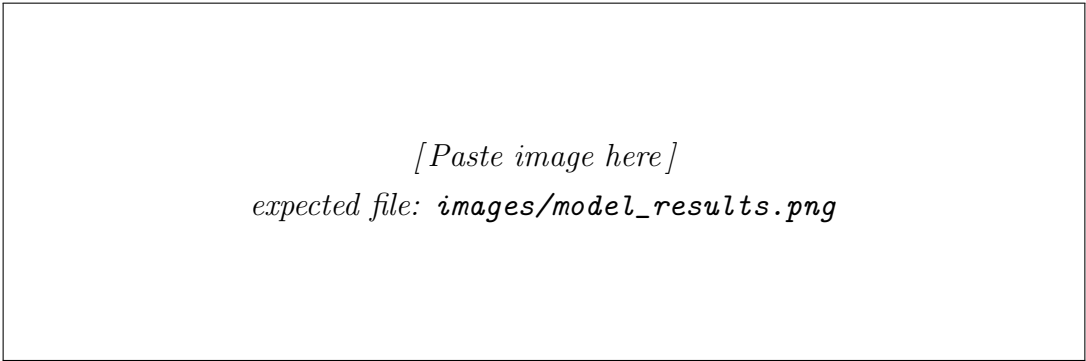


Figure 11: Model evaluation (water-quality comparison + confusion / feature importance)

7.3 Hardware Test Results

Sensor	Expected Range	Measured Range	Accuracy
pH	6.5-8.0	6.8-7.9	± 0.15 pH
DO	4-12 mg/L	4.2-11.8 mg/L	± 0.4 mg/L
Temp	20-30°C	20.5-29.8°C	± 0.6 °C
CO ₂	0-10 ppm	0.02-9.8 ppm	± 0.8 ppm

Table 24: Hardware Test Results

7.4 Known Issues

Issue	Impact	Workaround	Priority
Expo Go white screen (macOS)	Mobile app crashes	Clean metro cache, restart	P0 (Fixed)
Local LLM crashes during demos	Veronica unavailable	Migrated to OpenRouter cloud LLM	P1 (Fixed)
<code>react-native-reanimated</code> breaks Expo Go	Animations disabled	Use plain React Native animations	P2 (Won't fix)
Simulated data masks hardware bugs	False confidence	Set <code>SIMULATE_SENSORS=false</code>	P1 (Fixed)

Table 25: Known Issues

8 API & Technical Specifications

8.1 REST Endpoints (NestJS :3000)

8.1.1 Sensors

Method	Endpoint	Request	Response
GET	/sensors/latest	–	{ pH, temp_c, do_mg_l, CO2 }
GET	/sensors/:id/readings?range=24h	–	[{ readingId, value, timestamp }]
POST	/serial/reading	{ sensorId, type, value }	{ success: boolean }

Table 26: Sensors Endpoints

8.1.2 Actuators

Method	Endpoint	Request	Response
POST	/actuators/feed	{ cycles: 2 }	{ success: boolean }
POST	/actuators/pump	{ state: boolean }	{ success: boolean }
POST	/actuators/led	{ state: boolean }	{ success: boolean }
POST	/actuators/emergency-off	–	{ success: boolean }

Table 27: Actuators Endpoints

8.1.3 AI Predictor (Internal)

Method	Endpoint	Request	Response
POST	/predict/quality	{ pH, temp, do2, co2 }	{ score, status }
POST	/predict/disease	{ imagePath }	{ disease, confidence, bbox }
POST	/predict/count	{ imagePath }	{ count, confidence }
POST	/predict/movement	{ videoPath }	{ movementType, anomalyScore }
POST	/predict/attitude	{ videoPath }	{ tiltSeverity, anomalyScore }

Table 28: AI Predictor Endpoints

8.2 WebSocket Protocol

8.2.1 Event Format

Server → Client:

```

1 {
2   "event": "sensor:update",
3   "data": {
4     "readingId": 123,
5     "sensorId": 1,
6     "type": "pH",
7     "value": 7.12,
8     "unit": "pH",
9     "status": "ok",
10    "timestamp": "2026-06-06T04:51:00Z"
11  }
12 }
```

Client → Server:

```

1 {
2   "event": "command:feed",
3   "data": {
4     "cycles": 2
5   }
6 }
```

8.2.2 Server → Client Events (6 Total)

Event	Payload
sensor:update	SensorReading
alert:new	Alert
fish:count	FishCount
health:report	HealthReport
actuator:state	{ type, state }
hardware:status	{ online: boolean }

Table 29: WebSocket Events (Server to Client)

8.3 Hardware Interfaces

8.3.1 Serial Protocol (Arduino → Serial Bridge)

Baud Rate: 9600

Sensor Reading:

```

1 {
2   "sensorId": 1,
3   "type": "pH",
4   "value": 7.12,
5   "unit": "pH",
6   "status": "ok"
}
```



```
7 }
```

Actuator Command (Serial Bridge → Arduino):

```
1 CMD : FEED : 2  
2 CMD : PUMP : 1  
3 CMD : LED : 0  
4 CMD : EMERGENCY_OFF
```

9 Maintenance & Support

9.1 Monitoring

9.1.1 Logs

Service	Log Location	Level
Backend	services/backend/logs/	INFO
Serial Bridge	stdout	DEBUG
AI Predictor	stdout	INFO
Mobile App	Expo DevTools	DEBUG

Table 30: Log Locations

9.1.2 Health Checks

Service	Endpoint
Backend	GET /health → { status: 'ok' }
AI Predictor	GET /health → { status: 'ok' }
Serial Bridge	GET /health → { status: 'ok' }
Database	(TypeORM) → Connection successful

Table 31: Health Check Endpoints

9.2 Updates

9.2.1 Backend Update

```

1 cd services/backend
2 git pull origin develop
3 pnpm install
4 pnpm start:dev

```

9.2.2 Mobile App Update

```

1 cd apps/mobile
2 git pull origin develop
3 pnpm install
4 expo start --clear

```

9.2.3 AI Model Update

```

1 cd services/ai-predictor
2 git pull origin develop
3 # Replace model files in models/
4 uvicorn main:app --port 8001 --reload

```

9.3 Known Issues & Support

9.3.1 FAQ

Question	Answer
Why is the mobile app showing a white screen?	Clean metro cache: <code>rm -rf node_modules/.cache</code> then restart Expo.
Why are sensors not updating?	Check Arduino USB connection. Verify Serial Monitor shows JSON at 9600 baud.
Why is Veronica not responding?	Check <code>OPENROUTER_API_KEY</code> / connectivity. Reflexes still protect the tank; chat shows an offline fallback.
How do I calibrate the pH sensor?	Use pH 4.01, 7.01, 10.01 buffer solutions. See S6.4.3.
How do I export health reports?	Go to Fish AI screen → Settings → Export JSON.

Table 32: FAQ

9.3.2 Support Contacts

Role	Name	Responsibility
Lead Architect & AI Agent	Ismail (ismoiljon1101)	Backend, Mobile, Veronica, DevOps
Hardware	Sarvar	Arduino, Sensors, Actuators
ML Models	Firdavs	YOLO, Random Forest
Database	Maral	Supabase, ConvLSTM-VAE input

Table 33: Support Contacts

10 Appendices

10.1 Glossary

Term	Definition
DO	Dissolved Oxygen (mg/L)
pH	Potential of Hydrogen (0-14 scale)
CO ₂	Carbon Dioxide (ppm)
YOLO	You Only Look Once (real-time object detection)
ConvLSTM-VAE	Convolutional LSTM Variational Autoencoder (behavior analysis)
mAP	mean Average Precision (object detection metric)
MAE	Mean Absolute Error (regression metric)
AUC-ROC	Area Under ROC Curve (classification metric)
Expo Go	Expo's sandbox app for running React Native apps without building
Socket.IO	Real-time bidirectional event-based communication (WebSocket)
TypeORM	TypeScript ORM (Object-Relational Mapping)
NestJS	Progressive Node.js framework (inspired by Angular)
FastAPI	Modern Python web framework (based on Starlette + Pydantic)

Table 34: Glossary

10.2 Image Insertion Guide

Every figure in this document uses a `\figslot` that shows a labelled placeholder until the named file exists — so the PDF *always compiles*. To fill a figure, drop a file with the exact name below. Diagram renders go in `docs/`; screenshots and photos go in `docs/team/images/` (create that folder).

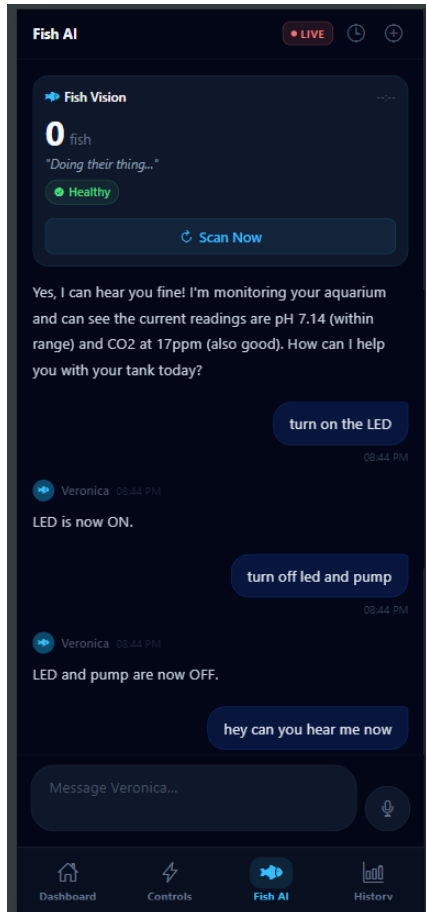
Expected file	Figure	How to obtain
<code>docs/architecture.png</code>	High-level architecture	Diagram render or drawn figure
<code>docs/ER_MODEL.png</code>	ER model	<code>mmdc -i docs/ER_MODEL.mmd -o docs/ER_MODEL.png</code>
<code>docs/WF_DIAGRAM.png</code>	Workflow diagram	<code>mmdc -i docs/WF_DIAGRAM.mmd -o docs/WF_DIAGRAM.png</code>
<code>images/veronica_chat.png</code>	Veronica chat	Mobile screenshot (Fish AI tab)
<code>images/veronica_confirm.png</code>	Confirm-before-act card	Mobile screenshot
<code>images/veronica_terminal.png</code>	Reasoning terminal	Mobile screenshot
<code>images/screen_dashboard.png</code>	Dashboard screen	Mobile screenshot
<code>images/screen_alerts.png</code>	Alerts screen	Mobile screenshot
<code>images/screen_controls.png</code>	Controls screen	Mobile screenshot
<code>images/hardware_build.png</code>	Hardware build	Photo of Arduino + sensors
<code>images/model_results.png</code>	Model evaluation	<code>docs/Water quality/results/*.png</code>

Table 35: Image Insertion Guide

Note: the `images/` paths are relative to `docs/team/` (where this `.tex` lives); `docs/renders` are reached via the configured `graphicspath`.

10.3 Mobile App Screenshots

The following screenshots show the mobile app in action. Portrait mobile screenshots are shown at 35% text width (preserving readability) with descriptive text beside them.



Veronica Chat Screen

Users chat with Veronica AI assistant. Type a message, and Veronica analyzes live sensor data, checks tank conditions, and can trigger actions (feed, LED, pump) with confirm-before-act.

Figure 12: Veronica AI Chat Interface



Confirm-Before-Act Card

When Veronica decides to take an action, she shows a card with the proposed action and reasoning. The user must confirm before any actuator changes the tank environment.

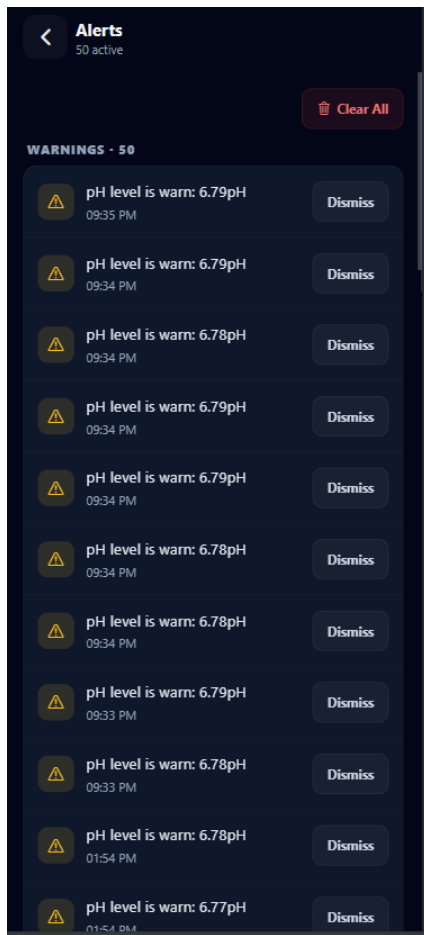
Figure 13: Veronica Confirm-Before-Act UI



Dashboard Screen

Live sensor cards show pH, temperature, DO₂, CO₂ with color-coded status badges. The health score hero gives an at-a-glance view of tank condition.

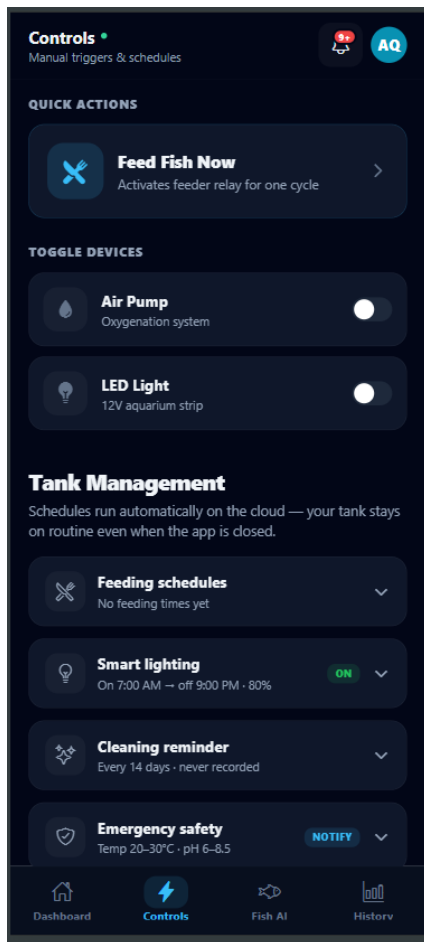
Figure 14: Mobile Dashboard — Live Sensor Data



Alerts Screen

Active alerts listed with severity (INFO / WARNING / CRITICAL / EMERGENCY). Swipe to acknowledge. Alert count badge on tab icon.

Figure 15: Mobile Alerts Screen



Controls Screen

Manual override for Feeder, Air Pump, and LED Strip. Emergency Stop button disables all actuators. Tank config and light/feed schedules managed here.

Figure 16: Mobile Controls — Actuator Override

10.4 References

Reference	Link
Repository	https://github.com/ismoiljon1101/capstone_aquarium_sejong
NestJS Documentation	https://docs.nestjs.com
Expo Documentation	https://docs.expo.dev
FastAPI Documentation	https://fastapi.tiangolo.com
TypeORM Documentation	https://typeorm.io
Socket.IO Documentation	https://socket.io/docs
YOLOv8 Documentation	https://docs.ultralytics.com
ConvLSTM-VAE Paper	https://arxiv.org/abs/2003.06565
Supabase Documentation	https://supabase.com/docs

Table 36: References

10.5 Changelog

Version	Date	Changes
v1.0	2026-05-15	Initial documentation (basic structure)
v1.5	2026-06-05	Added ER models, expanded API specs
v2.0	2026-06-06	Complete IEEE-inspired restructure, added ER diagrams, expanded all sections
v3.0	2026-06-06	Corrected ownership (129/152), rewrote Veronica with the real 17-tool agent + deterministic safety layer, real cron jobs, image slots, brand fix (Fishlinic)

Table 37: Changelog

Document Approval

Role	Name	Signature	Date
Lead Architect	Ismoiljon (ismoiljon1101)	☐	2026-06-06
Team Member	Maral	☐	–
Team Member	Sarvar	☐	–
Team Member	Firdavs	☐	–
Institution	Sejong University	☐	–

This document is the complete technical documentation for the Fishlinic smart aquarium monitoring system.

For questions or clarifications, contact the Lead Architect (ismoiljon1101@gmail.com).